MILESTONE 1 - NARRATIVE FLOW CO WRITER
MODULE 1 - FRONT END

Page 1: Title & Executive Summary
Project Title: Narrative Flow – AI Story Co-Writer
Framework: Streamlit (Python)

1.1 Overview

Narrative Flow is an immersive, genre-adaptive storytelling environment. It serves as a creative sanctuary where users can draft narratives with AI assistance. The application focuses on three core pillars:

- **Immersive Experience:** Genre-based dynamic UI styling.
- **Workflow Efficiency:** Mode-driven AI responses (Fast, Thinking, Creative).
- **Persistence:** Local JSON-based storage for multi-draft management.

2: System Architecture & Technologies

2.1 High-Level Architecture

The application follows a modular "State-Storage-Style" pattern:

- **The UI Engine:** Built on Streamlit, handling sidebar controls and the main narrative flow.
- **The State Manager:** Utilizes st.session_state to maintain variables across reruns, preventing data loss during interaction.
- **The Data Layer:** A JSON persistence engine that reads and writes to chat_history.json.
- **Performance:** Utilises single code file structure

2.2 Technology Stack

- **Stream lit:** Primary frontend framework.
- **Python-Docx:** Library used for exporting manuscripts to Word format.
- **UUID:** For generating unique identifiers for every story draft.
- **CSS/HTML:** Custom injection for glass morphism and background switching.

3: Theming & Visual Engineering

3.1 Dynamic Genre Theming

One of the application's unique features is the Genre Selector, which triggers a complete UI overhaul.

- **Fantasy:** Thematic imagery and colour palettes suited for magic and lore.

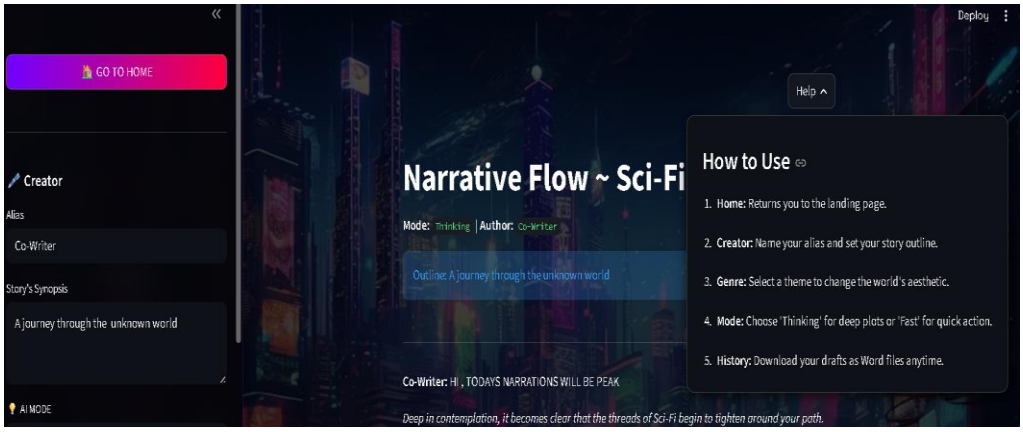
- **Sci-Fi:** Futuristic cityscapes and neon-hued aesthetics.
- **Thriller/Horror:** Darker, high-contrast themes to set a tense mood.

3.2 CSS Injection Logic

The apply styles () function handles the following visual components:

- **Glass morphism:** Using backdrop-filter for translucent chat bubbles.
- **Animated Transitions:** Smooth reveal effects for new text.
- **Custom Typography:** Integration of *Syncopate* and *Space Grotesk* fonts.

UI DESIGN



4: Functional Logic & AI Modes

4.1 Narrative AI Modes

The user can select from three "AI Modes" that influence the tone and structure of the generated narrative:

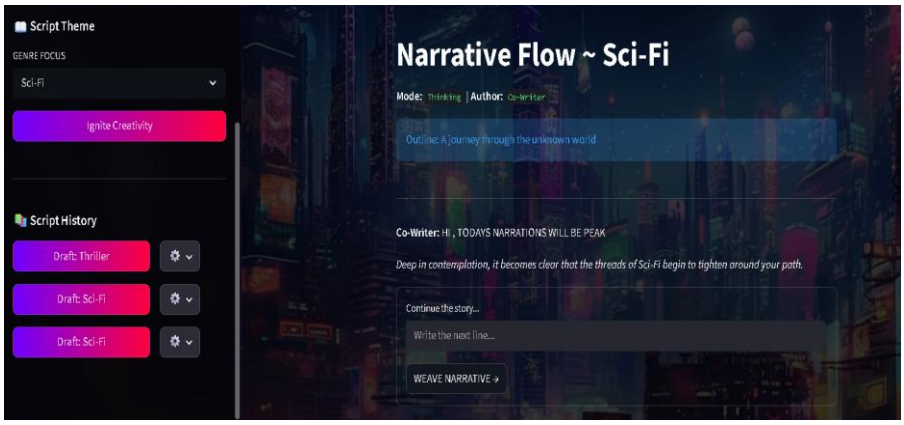
- **Fast:** Direct and action-oriented transitions (Prefix: "Suddenly,").
- **Thinking:** Reflective and philosophical tone.
- **Creative:** Dream-like and highly descriptive prose.

4.2 The Chat Processing Loop

When a user submits a prompt, the following logic occurs:

- The text is captured and appended to the session conversation.
- The system applies a mode-based prefix to guide the AI's "thought process".

- The final response is rendered using the genre-specific CSS classes.



5: Persistence

5.1 JSON Schema

To ensure users can return to their drafts, data is saved in a structured JSON

- **Conversation ID:** A unique UUID.
- **Metadata:** Stores the genre, mode, author alias, and story synopsis.
- **Message Array:** A list of dictionaries containing "role" (user/assistant) and "content".

5.2 Storage Engine

- **load_data():** Safely opens the JSON file and handles potential corruption via try/except blocks.
- **save_data():** Serializes the current session state and writes it back to the local disk.

6: Export, Security, & Future Roadmap

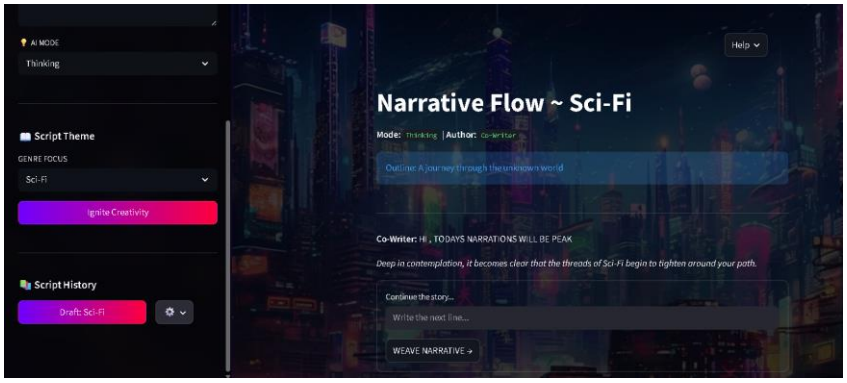
6.1 Document Export Engine

Users can export their complete narrative as a professional manuscript:

- **Format:** .docx file.
- **Layout:** Includes a bold title, author alias, and distinct styling for AI narration (Italics) versus user input (Bold).

6.2 Error Handling & Security

- **Session Validation:** Ensures a draft is selected before rendering the chat interface.
- **Data Integrity:** Implements fallback mechanisms if the JSON storage is missing or unreadable.



6.3 Future Opportunities

- Integration with live LLM APIs (Gemini/OpenAI).
- User authentication for multi-user cloud support.
- Real-time collaboration tools for co-writing.

By
R.V. Aswath Narayanan
aswathnarayanan48@gmail.com